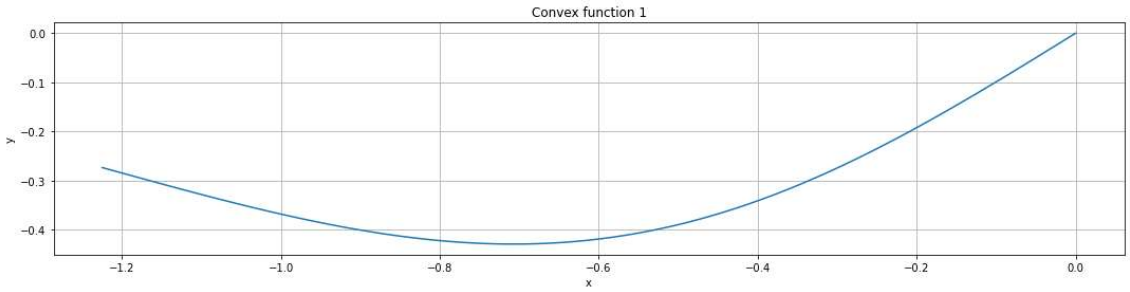
 # Experiment 2(a):

```
import numpy as np
import matplotlib.pyplot as plt
# Function 1: x * e^(-x^2)
def convex_func_1(x):
    return x * np.exp(-x**2)
# Gradient of Function 1:
def gradient_convex_func_1(x):
    return np.exp(-x**2) - 2 * x**2 * np.exp(-x**2)
# Generate data points
x_data = np.linspace(-np.sqrt(1.5), 0, 100)
y_data = convex_func_1(x_data)
# Visualization for Function 1
plt.figure(figsize=(15, 4))
plt.plot(x_data, y_data)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Convex function 1')
plt.grid()
plt.tight_layout()
plt.show()
# Specification of parameters
stopping_threshold = 1e-3  # Adjust this threshold as needed
step_size = [0.001, 0.005, 0.01, 0.05]
x_init = -np.sqrt(1.5)
print('[Step Size, No. of iterations, [xmin], fxmin] =')
print('\n')

# Run algorithm with different step_size
for sz in step_size:
    num_iterations = 0
```

```
    x_values = x_init
    gradient = gradient_convex_func_1(x_values)
        # Gradient Descent Algorithm with stopping threshold and iteration count
    while np.linalg.norm(gradient) >= stopping_threshold:
        x_new = x_values - sz * gradient
        x_values = x_new
        gradient = gradient_convex_func_1(x_values)
        num_iterations += 1
    # Print the results
    results=[sz,num_iterations,x_values,convex_func_1(x_values)]
    print(results)
    print('\n')
```

#Results:



[Step Size, No. of iterations, [xmin], fxmin] =

[0.001, 4322, -0.7076890245937736, -0.4288816517719241]

[0.005, 862, -0.7076888034331911, -0.4288816519926985]

[0.01, 430, -0.7076834858737564, -0.42888165727572897]

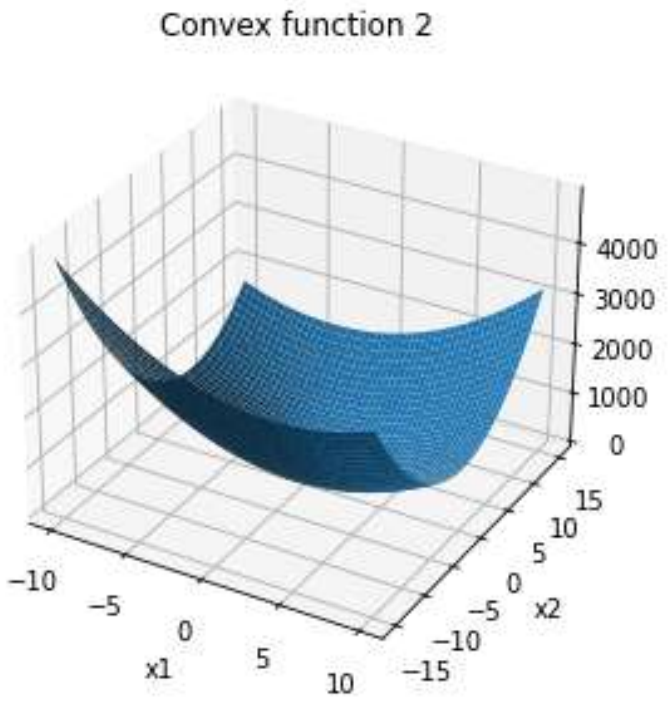
[0.05, 84, -0.7076593244182173, -0.4288816806698405]

 # Experiment 2(b):

```
import numpy as np
import matplotlib.pyplot as plt

# Function 2: 10*x1^2 + 5*x1*x2 + 10*(x2 - 3)^2
def convex_func_2(x):
    x1, x2 = x
    return 10 * x1**2 + 5 * x1 * x2 + 10 * (x2 - 3)**2

# Gradient of Function 2:
def gradient_convex_func_2(x):
    x1, x2 = x
    grad_x1 = 20 * x1 + 5 * x2
    grad_x2 = 5 * x1 + 20 * (x2 - 3)
    return np.array([grad_x1, grad_x2])

# Generate data points
x1_data = np.linspace(-10, 10, 300)
x2_data = np.linspace(-15, 15, 300)
x =[x1_data, x2_data]

# Visualization for Function 1
fig = plt.figure(figsize=(15, 4))
x1, x2 = np.meshgrid(x1_data, x2_data)
ax = fig.add_subplot(projection='3d')
ax.plot_surface(x1, x2, convex_func_2(np.array([x1, x2])))
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Convex function 2')
plt.grid()
plt.tight_layout()
```

```
plt.show()


# Specification of parameters
stopping_threshold = 1e-3  # Adjust this threshold as needed
step_size = [0.001, 0.005, 0.01, 0.05]
x_init =np.array([10,15])

print('[Step Size, No. of iterations, [xmin], fxmin] =')
print('\n')

# Run algorithm with different step_size

for sz in step_size:
    num_iterations = 0
    x_values = x_init
    gradient = gradient_convex_func_2(x_values)
    # Gradient Descent Algorithm with stopping threshold and iteration coun
    while np.linalg.norm(gradient) >= stopping_threshold:
        x_new = x_values - sz * gradient
        x_values = x_new
        gradient = gradient_convex_func_2(x_values)
        num_iterations += 1

    # Print the results
    results=[sz,num_iterations,x_values,convex_func_2(x_values)]
    print(results)
    print('\n')
```

#Results:



[Step Size, No. of iterations, [xmin], fxmin] =

[0.001, 614, array([-0.80004464, 3.20004865]), -5.999999967261684]

[0.005, 119, array([-0.80004534, 3.20004818]), -5.999999967157968]

[0.01, 58, array([-0.80003966, 3.20004094]), -5.999999975632579]

[0.05, 10, array([-0.7999897 , 3.20001125]), -5.999999997093255]